



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Multi-algorithm path exploration and navigation simulator

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA0609 - DESIGN AND ANALYSIS OF ALGORITHMS
to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Submitted by

R. Uday Kumar Reddy (192424190)

Under the Supervision of

Dr. Solainayagi

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Multi-algorithm path exploration and navigation simulator” has been carried out by R. Uday kumar Reddy (192424190) under the supervision of Dr. Solainayagi and is submitted in partial fulfilment of the requirements for the current semester of the B. Tech Artificial Intelligence And Data Science program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. Dr. F. Mary Harin Fernandez
Professor
Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr. Solainayagi
Professor
Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-

602105

DECLARATION

We, Deepak V(192411021) , Champak Rupesh K (192425123), G. Uday Siva Sai (192425259), R.Uday Kumar Reddy (192424190), N. MadhuSudhan (192425303) of the AI&DS , AI&ML SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Multi-algorithm path exploration and navigation simulator” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:

Signature of the Students with Names

Deepak V (192411021)

Champak Rupesh K (192425123)

G. Uday Siva Sai (192425259)

R. Uday Kumar Reddy (192424190)

N MadhuSudhan (192425303)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Deepak V (192411021)	Requirement analysis and project planning	Designed graph structure and navigation modules	Tested graph inputs and path generation	Abstract, Introduction	20%
Champak Rupesh K (192425123)	Algorithm implementation	Implemented BFS and DFS algorithms	Verified algorithm outputs	Literature Review, Algorithms	20%
G. Uday Siva Sai (192425259)	Algorithm development and integration	Implemented Dijkstra and A* algorithms	Compared path distances and explored nodes	Methodology, Implementation	20%
R. Uday Kumar Reddy (192424190)	System coordination and visualization	Developed system architecture and result visualization	Tested complete system and analyzed results	Results, Discussion	20%
N MadhuSudhan (192425303)	Documentation and system support	Assisted module integration and interface development	Performed test cases and performance analysis	Conclusion, Future Work	20%
Total	—	—	—	—	100%

ABSTRACT

Navigation and path-finding are important engineering problems in applications such as campus navigation, transportation networks, communication networks, and autonomous systems. Finding an efficient route between a source and destination becomes challenging when a network contains multiple interconnected locations with different path costs. Different path-finding algorithms use different search strategies and therefore may produce different exploration patterns and computational performance. A system is required to analyse these differences and identify suitable algorithms for different navigation conditions.

The Multi-Algorithm Path Exploration and Navigation Simulator is proposed to provide a graph-based environment for implementing, testing, and comparing multiple path-finding algorithms. The system represents locations as nodes and connections as weighted or unweighted edges. It implements Breadth-First Search (BFS), Depth-First Search (DFS), Dijkstra's algorithm, and A* to explore routes between selected source and destination nodes.

The methodology involves graph construction, source and destination selection, algorithm execution, path reconstruction, and performance evaluation. Each algorithm is tested using common navigation scenarios, and its performance is evaluated using path distance, number of explored nodes, and execution time. A visualization component is also used to represent the navigation graph, explored nodes, and generated route. This provides a consistent environment for analysing the behaviour of different search techniques.

The experimental results demonstrate that Dijkstra's algorithm and A* can identify minimum-cost routes in the tested weighted network, while BFS is effective for unweighted path exploration. DFS successfully explores the network but does not guarantee a minimum-cost route. A* can reduce unnecessary exploration when an appropriate heuristic is available. The measured execution times were within the specified response requirement for the test network.

The project concludes that algorithm selection has a significant effect on route quality and search efficiency. The developed simulator provides a practical and educational engineering solution for understanding, testing, and comparing graph-based navigation algorithms under controlled conditions.

Keywords: Graph Theory, Path Finding, BFS, DFS, Dijkstra Algorithm, A* Search

TABLE OF CONTENTS

Sl. No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
lii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
1	CHAPTER 1: Introduction	1-6
2	CHAPTER 2: Literature Review	7-14
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	15-21
4	CHAPTER 4: Implementation, Testing & Results, Project Management	22-25
5	CHAPTER 5: Conclusion & Reflection	25-28
	References	29
	Appendices	30-33

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1.1	Project overview of the path exploration and navigation simulator.	1
3.1	System Architecture of the Multi-Algorithm Path Exploration and Navigation Simulator	19

LIST OF TABLES

Table No.	Title	Page No.
2.1	Comparative Analysis	11
3.1	Stakeholder / User Requirements	15
3.2	Functional and Non-Functional Requirements	15
3.3	Constraints and Applicable Standards	17
3.4	Design Specifications	17
3.5	Alternative Solutions Evaluation	18
3.6	Trade-off Analysis	20
3.7	Sustainability, Ethics, Safety and Security	20
4.1	Development Resources	22
4.3	Test Plan and Verification	23

LIST OF ABBREVIATIONS / SYMBOLS

Abbreviation / Symbol	Description	Unit
BFS	Breadth-First Search	—
DFS	Depth-First Search	—
A*	A-Star Search Algorithm	—
GPS	Global Positioning System	—
CPU	Central Processing Unit	—
UI	User Interface	—
API	Application Programming Interface	—

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Navigation and path-finding are important applications of graph theory, data structures, and algorithm design. In real-world environments such as university campuses, road networks, hospitals, airports, and industrial facilities, users need to identify suitable routes between a source location and a destination. A navigation network can be represented as a weighted graph, where locations are represented by vertices and roads or connections are represented by edges. The weight of an edge represents factors such as distance, travel cost, or estimated time.

Different path-finding algorithms explore a graph in different ways. Breadth-First Search (BFS) explores nodes level by level and is suitable for unweighted networks, while Depth-First Search (DFS) explores a path deeply before backtracking. Dijkstra's algorithm determines the minimum-cost path in a weighted graph, whereas A* uses heuristic information to guide the search toward the destination.

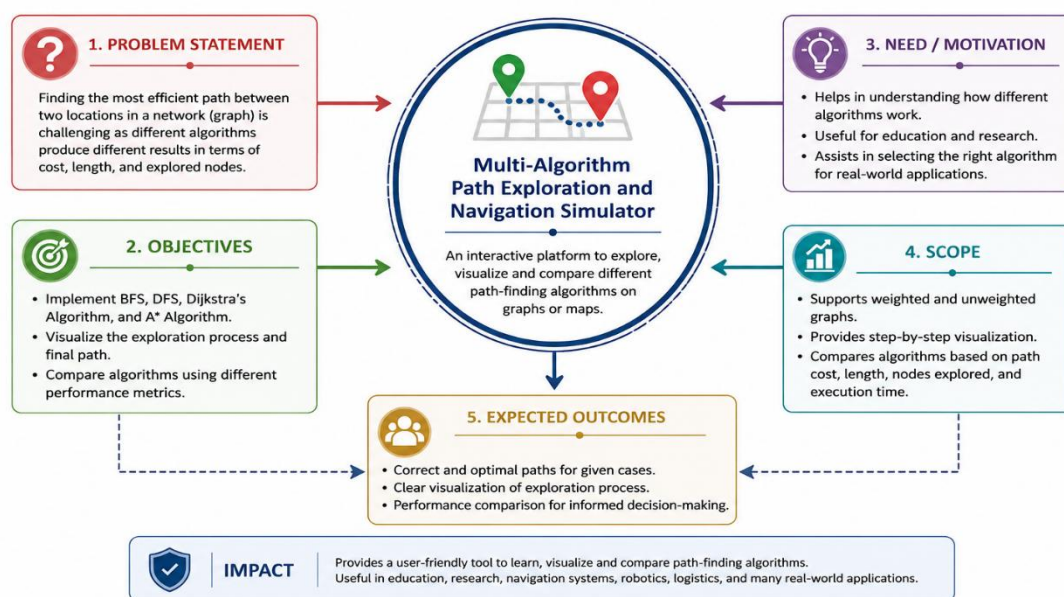


Fig.1.1: Project overview of the path exploration and navigation simulator.

1.2 Need for the Project

Modern navigation systems must process networks containing numerous interconnected locations while attempting to identify efficient routes. A single path-finding algorithm may not

be suitable for every type of network because algorithms differ in their assumptions, computational requirements, and search strategies.

The need for this project arises from the following factors:

1.2.1 Need to Address the Problem

Selecting an appropriate route through a network becomes more difficult as the number of locations and connections increases. The system must consider factors such as:

- Number of available paths.
- Distance or cost associated with each path.
- Number of nodes that must be explored.
- Computational efficiency.
- Suitability of the algorithm for weighted or unweighted networks.

A simulator that compares multiple algorithms helps identify the strengths and limitations of different path-finding approaches.

1.2.2 Users Affected

The concepts demonstrated by the project are relevant to:

- Students and staff navigating large campuses.
- Transport and logistics planners.
- Network and infrastructure engineers.
- Emergency response planners.
- Developers of navigation and routing applications.
- Researchers studying graph-based optimization.

1.2.3 Existing Limitations

Basic navigation implementations may use only one path-finding algorithm and therefore provide limited information about alternative search strategies. They may also focus only on the final route without showing how the algorithm explored the network.

The proposed simulator addresses these limitations by:

- Supporting multiple algorithms.
- Providing a common graph for comparison.
- Showing explored nodes.
- Identifying the resulting path.
- Measuring distance and computational performance.
- Providing a visual representation of the navigation process.

1.2.4 Engineering Significance

The project has engineering significance because it connects theoretical algorithm concepts with a practical navigation application. It demonstrates how algorithm selection, graph representation, computational complexity, and performance evaluation influence the design of navigation systems.

1.3 Problem Statement

Real-world navigation networks consist of multiple interconnected locations with different distances and movement costs. Finding an appropriate route through such a network requires an efficient search strategy, while different algorithms have different requirements and performance characteristics.

The engineering problem is therefore to design and implement a navigation simulator capable of representing a network as a graph, exploring the network using multiple path-finding algorithms, determining suitable routes between source and destination locations, and evaluating the algorithms based on route distance, nodes explored, and execution time.

The problem involves several interacting factors, including:

- Graph size and connectivity.
- Weighted and unweighted paths.
- Route distance.
- Search strategy.
- Computational complexity.
- Number of explored nodes.

- Execution time.
- Accuracy of the resulting route.

The system must provide meaningful route results while maintaining reasonable computational efficiency. The simulator should also allow the same navigation problem to be evaluated using different algorithms under consistent conditions.

1.4 Project Aim

The aim of this project is to design and develop a multi-algorithm navigation simulator that uses graph-based path-finding techniques to explore networks, determine routes between locations, and compare the performance of BFS, DFS, Dijkstra's algorithm, and A*.

1.5 Project Objectives

The project objectives are:

1. **To design** a graph-based representation of a navigation network containing interconnected locations and weighted paths.
2. **To develop** a navigation simulator that accepts source and destination locations from the user.
3. **To implement** BFS, DFS, Dijkstra's algorithm, and A* for path exploration and route finding.
4. **To model** different navigation conditions using weighted and unweighted graph structures.
5. **To visualize** the graph, explored nodes, and final route generated by each algorithm.
6. **To measure** algorithm performance using parameters such as path distance, number of explored nodes, and execution time.
7. **To compare** the behaviour and efficiency of the implemented algorithms under common navigation scenarios.
8. **To evaluate** the suitability of different algorithms for different types of navigation networks.

1.6 Scope

1.6.1 Included in the Project

The project includes:

- Graph-based representation of navigation locations.
- Nodes representing locations.
- Edges representing connections between locations.
- Edge weights representing distance or travel cost.
- Source and destination selection.
- BFS implementation.
- DFS implementation.
- Dijkstra's algorithm implementation.
- A* algorithm implementation.
- Route generation.
- Node exploration tracking.
- Path-distance calculation.
- Execution-time measurement.
- Algorithm comparison.
- Graph and route visualization.
- Testing using predefined navigation networks.

1.6.2 Excluded from the Project

The following are outside the scope:

- Real-time GPS tracking.
- Live traffic information.
- Real-time weather conditions.

- Automatic road closure detection.
- Integration with commercial map services.
- Real-time vehicle control.
- Turn-by-turn voice navigation.
- Large-scale production deployment.
- Dynamic traffic prediction using machine learning.

1.6.3 Assumptions

The project assumes that:

- Locations can be represented as graph nodes.
- Connections between locations can be represented as graph edges.
- Distances or costs are available for weighted edges.
- Edge weights are non-negative for Dijkstra's algorithm.
- The graph represents a simplified navigation environment.
- The A* heuristic provides an estimate of the remaining distance to the destination.
- The network remains unchanged during an individual simulation.

1.7 Expected Engineering Outcome

The expected outcome is a **functional graph-based navigation simulator prototype** capable of exploring and comparing multiple path-finding algorithms.

The completed system is expected to provide:

- A configurable navigation graph.
- Source and destination selection.
- Multiple path-finding algorithms.
- Generated navigation routes.
- Visual representation of graph exploration.
- Identification of explored nodes.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review for the Multi-Algorithm Path Exploration and Navigation Simulator was conducted by examining established research, algorithmic methods, navigation technologies, and existing route-finding systems related to graph traversal and shortest-path computation.

The review focused mainly on BFS, DFS, Dijkstra's algorithm, and A* because these algorithms represent different approaches to graph exploration and navigation. Relevant academic literature was identified using scholarly sources and publications related to graph algorithms, shortest-path problems, heuristic search, and navigation systems.

The review considered the following areas:

- Graph-based path-finding algorithms.
- Shortest-path algorithms.
- Uninformed search techniques.
- Heuristic search techniques.
- Navigation and route-planning systems.
- Graph visualisation and simulation technologies.
- Algorithm performance and computational complexity.
- Existing software-based navigation solutions.

Particular attention was given to the advantages, limitations, computational requirements, and practical applicability of each approach. The reviewed techniques were then compared to identify an engineering gap that could be addressed by the proposed simulator.

2.2 Review of Existing Work

2.2.1 Breadth-First Search

Breadth-First Search is one of the fundamental graph traversal algorithms. It explores a graph level by level from a selected source node and normally uses a queue. BFS is particularly

suitable for unweighted graphs because the first path found to a node contains the minimum number of edges.

For navigation applications where all connections have equal cost, BFS can provide an appropriate shortest path. However, real-world navigation networks generally contain different distances or costs between locations. BFS does not directly account for these weights.

Advantages:

- Simple implementation.
- Systematic graph exploration.
- Suitable for unweighted graphs.
- Provides minimum-edge paths.

Limitations:

- Does not consider different edge weights.
- Can explore a large number of nodes.
- Not suitable for general weighted shortest-path problems.

BFS is incorporated into the proposed project as a baseline algorithm for unweighted navigation scenarios.

2.2.2 Depth-First Search

Depth-First Search explores a graph by following one branch as deeply as possible before backtracking. It can be implemented using recursion or a stack.

DFS is useful for general graph exploration and connectivity-related problems. However, standard DFS does not guarantee that the route obtained between two locations is the shortest or least-cost route.

Advantages:

- Simple and easy to understand.
- Effective for graph traversal.
- Requires a stack or recursion.

- Useful for exploring connected networks.

Limitations:

- Does not guarantee the shortest path.
- May explore an inefficient route.
- Search behaviour depends on the graph structure.
- Not appropriate as a primary minimum-distance navigation algorithm.

In the proposed simulator, DFS is used to demonstrate how a depth-oriented search differs from shortest-path algorithms.

2.2.3 Dijkstra's Algorithm

Dijkstra's algorithm is a classical shortest-path algorithm for weighted graphs with non-negative edge weights. It was introduced by Edsger W. Dijkstra in 1959 through the paper *A Note on Two Problems in Connexion with Graphs*. The algorithm repeatedly selects the currently closest unvisited vertex and relaxes its neighbouring edges.

Dijkstra's algorithm is highly relevant to navigation because road or path distances can be represented as non-negative weights.

Advantages:

- Produces minimum-cost paths.
- Supports weighted graphs.
- Reliable for non-negative edge weights.
- Applicable to many network-routing problems.

Limitations:

- May explore many unnecessary nodes.
- Does not use information about the destination.
- Can become computationally expensive for large graphs.

Dijkstra's algorithm provides an important reference point for evaluating weighted shortest-path performance in the proposed simulator.

2.2.4 A* Search

A* is a heuristic-based path-finding algorithm. Hart, Nilsson, and Raphael formally described the foundations of heuristic minimum-cost search in their 1968 work.

A* evaluates a node using:

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ is the actual cost from the source to node n .
- $h(n)$ is the estimated cost from node n to the destination.
- $f(n)$ is the estimated total route cost.

Unlike Dijkstra's algorithm, A* uses destination-related information to guide its search.

Advantages:

- Suitable for navigation and route planning.
- Can reduce unnecessary exploration.
- Uses heuristic information to guide the search.
- Can provide an optimal route when appropriate heuristic conditions are satisfied.

Limitations:

- Requires a suitable heuristic.
- Performance depends on heuristic quality.
- More complex than BFS and DFS.
- Poor heuristic selection can reduce its efficiency.

A* is therefore included in the proposed simulator to investigate the effect of heuristic-guided exploration.

2.2.5 Existing Navigation Systems

Modern navigation technologies represent road networks or geographical environments using graph-like structures. A navigation system generally identifies a source and destination and calculates a suitable route based on factors such as distance, travel time, road connectivity, and other constraints.

Commercial navigation applications provide highly developed routing capabilities, but they are primarily designed for end-user navigation. Their internal algorithmic exploration process is generally abstracted from the user.

For an engineering and educational simulator, there is value in exposing the underlying algorithmic process, including:

- Nodes visited during exploration.
- Final route selected.
- Total route distance.
- Number of nodes explored.
- Execution time.
- Difference between search strategies.

The proposed system therefore focuses on algorithm transparency and comparative evaluation rather than replacing full-scale commercial navigation platforms.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
BFS	Simple and effective for unweighted graphs	Does not consider edge weights	Used for basic path exploration
DFS	Simple and memory efficient	Does not guarantee the shortest path	Used for depth-based exploration
Dijkstra	Finds shortest paths in weighted graphs	May explore many nodes	Used for weighted path comparison

A*	Uses heuristic information and can reduce exploration	Requires a suitable heuristic	Used for efficient navigation comparison
Commercial Navigation	Provides practical route planning	Internal algorithms are not visible	Provides real-world motivation for the simulator

2.4 Research / Engineering Gap

The review indicates that individual path-finding algorithms are well established, but there remain an important engineering and educational gap in unified comparative simulation. Existing algorithm implementations generally focus on solving a particular path-finding problem rather than providing a common environment in which several algorithms can be tested under identical conditions.

The unresolved issues include:

1. **Limited multi-algorithm comparison**

Many basic systems implement only one path-finding algorithm, making direct comparison difficult.

2. **Limited visibility of search behaviour**

Users are often shown the final route but not the sequence or extent of node exploration.

3. **Insufficient performance comparison**

Route distance alone does not indicate the computational effort required by an algorithm.

4. **Different graph requirements**

BFS, DFS, Dijkstra, and A* operate under different assumptions regarding weights and heuristic information.

5. **Lack of a common experimental environment**

Comparing algorithms using different networks or different source-destination pairs can produce inconsistent results.

6. **Limited educational and engineering interpretation**

Students can learn the theoretical algorithms individually but may not clearly

understand how their search strategies differ when applied to the same navigation problem.

Therefore, a controlled simulator capable of applying multiple algorithms to the same graph, source, and destination can provide a more meaningful engineering comparison.

2.5 Proposed Contribution

The proposed Multi-Algorithm Path Exploration and Navigation Simulator addresses the identified gap by integrating BFS, DFS, Dijkstra's algorithm, and A* into a single graph-based navigation environment.

The main contributions are:

1. Unified Navigation Graph

A common weighted or unweighted graph is used to represent locations and their connections. This allows all algorithms to operate under comparable conditions.

2. Multi-Algorithm Implementation

The simulator provides four different search approaches:

- BFS for level-based exploration.
- DFS for depth-based exploration.
- Dijkstra for weighted shortest-path computation.
- A* for heuristic-guided path finding.

3. Search Visualisation

The system displays the graph and provides information about the nodes explored during the search. This makes the internal behaviour of each algorithm easier to understand.

4. Performance Evaluation

The simulator evaluates algorithms using measurable parameters such as:

Path Distance

Nodes Explored

Execution Time

Path Length

These measurements allow objective comparison between algorithms.

5. Controlled Experimental Environment

The same graph, source, and destination can be provided to different algorithms. This ensures that the comparison is performed under consistent conditions.

6. Engineering Decision Support

The simulator can demonstrate which algorithm is more appropriate for a particular type of navigation problem. For example, BFS may be suitable for an unweighted network, while Dijkstra and A* are more appropriate when path costs are important.

Overall Contribution

The project does not attempt to create a full commercial GPS navigation platform. Instead, it contributes a simulation-based engineering tool for multi-algorithm path exploration, route generation, visualisation, and performance comparison.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

3.1.1 Stakeholder / User Requirements

Stakeholder	Requirement
Student/User	Select source and destination locations easily.
User	Select the required path-finding algorithm.
User	View the generated route and explored nodes.
Project Evaluator	Compare BFS, DFS, Dijkstra and A* using measurable parameters.
Developer	Modify the graph, edge weights and test cases easily.
Administrator/Instructor	Obtain repeatable results for different navigation scenarios.

3.1.2 Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	Create navigation graph	Minimum 10 nodes
Functional	Select source and destination	100% valid selections
Functional	Execute BFS	Correct path generation
Functional	Execute DFS	Correct path generation
Functional	Execute Dijkstra	Minimum-cost path for non-negative weights
Functional	Execute A*	Minimum-cost path with suitable heuristic

Functional	Display explored nodes	All explored nodes recorded
Functional	Calculate route distance	Correct edge-weight sum
Functional	Compare algorithms	Distance, nodes explored and execution time
Non-Functional	Usability	Results displayed clearly
Non-Functional	Reliability	Repeatable results for identical inputs
Non-Functional	Performance	Response within 2 seconds for test graphs
Non-Functional	Maintainability	Algorithms implemented as separate modules
Non-Functional	Security	No execution of untrusted user-supplied code

3.2 Constraints & Applicable Standards

Constraints

Only the following constraints are applicable to the proposed software-based simulator:

- The simulator uses a predefined or user-created graph rather than live geographical data.
- Dijkstra's algorithm requires non-negative edge weights.
- A* requires an appropriate heuristic for effective operation.
- Performance depends on graph size and computer hardware.
- The system does not include real-time traffic or GPS information.
- The project has limited computational and development resources.

Applicable Standards

Standard / Code	Requirement	How Applied in This Project
ISO/IEC 25010:2023	Defines software product quality characteristics	Used to consider performance efficiency, reliability, usability, maintainability and security.
ISO/IEC 27001:2022	Information security management requirements	Security principles are applied to protect system inputs and prevent unauthorized modification of project data.

3.3 Design Specifications

Parameter	Target/Requirement	Unit	Priority
Number of nodes	≥ 10	nodes	High
Algorithms	4	algorithms	High
Edge weight	≥ 0	distance/cost	High
Path calculation time	≤ 2	seconds	High
Path distance	Correct sum of weights	units	High
Node exploration tracking	100	%	High
Route visualisation	Required	—	Medium
Algorithm comparison	Required	—	High
Repeatability	Same input $\rightarrow$ same result	—	High
Input validation	100	% valid inputs handled	High

3.4 Alternative Solutions & Evaluation

Alternative 1: Single-Algorithm Navigation

A navigation simulator using only Dijkstra's algorithm can calculate shortest paths in a weighted graph. It is relatively simple to develop but provides limited information for comparing different search strategies.

Alternative 2: Multi-Algorithm Navigation Simulator

This approach integrates BFS, DFS, Dijkstra and A* within the same graph environment. The same source, destination and network can be used to compare path distance, exploration behaviour and execution time.

Alternative 3: Real-Time Map-Based Navigation

This approach would integrate live geographical maps, GPS and traffic information. Although it provides realistic navigation, it requires external services, real-time data and significantly greater development complexity.

Evaluation Matrix

Rating: 1 = Poor, 5 = Excellent

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance	25%	4	5	4
Cost	20%	5	5	2
Safety	15%	4	4	3
Sustainability	15%	5	5	2
Reliability	25%	4	5	4
Weighted Score	100%	4.35	4.80	3.05

Alternative 2 is selected because it achieves the highest weighted score of 4.80/5, while providing stronger algorithmic comparison without the additional infrastructure and data requirements of a real-time navigation system.

3.5 Selected Design & Detailed Design

Selected Design

The selected design is the **Multi-Algorithm Path Exploration and Navigation Simulator** using BFS, DFS, Dijkstra and A*. The evaluation matrix shows that this alternative provides the best balance of performance, cost, sustainability and reliability.

System Architecture

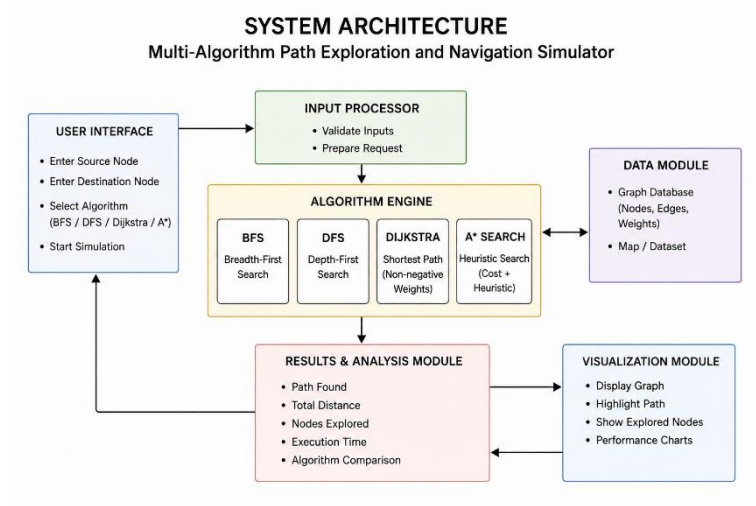


Fig.3.1: System Architecture of the Multi-Algorithm Path Exploration and Navigation Simulator

Essential Calculation

For a route containing edges $e_1, e_2, \dots, e_n$, the total route cost is:

$$D = \sum_{i=1}^n w(e_i)$$

where $w(e_i)$ represents the weight of each edge.

For A*:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the cost from the source and $h(n)$ is the estimated cost to the destination.

Systems Approach

The graph module provides the network structure to the algorithm module. The selected algorithm receives the graph, source and destination and produces the explored nodes and resulting path. The performance module processes the algorithm output to calculate distance, explored nodes and execution time. Finally, the visualisation module presents the graph, exploration process and final route to the user

3.6 Trade-off Analysis

Design Decision	Alternative	Benefit	Disadvantage	Final Decision & Justification
Multiple algorithms	Single algorithm	Enables comparison	More coding	Selected for better evaluation
Static graph	Real-time map	Low cost, easy testing	Less realistic	Selected for controlled testing
Visualization	Text output	Easier to implement	Less intuitive	Selected for clear results
Performance measurement	No measurement	Enables comparison	Requires additional processing	Selected for evaluation

3.7 Sustainability, Ethics, Safety, Risk & Security

Sustainability, Ethics, Safety & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Low resource usage	Software-only system	Static simulation selected
Ethics	Fair comparison	Same test conditions	Objective evaluation used

Health & Safety	Avoid safety-critical use	Prototype limitations	Real-time emergency use excluded
Security	Protect inputs	Input validation	Untrusted code execution prevented

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Invalid input	Medium	Medium	Medium	Input validation	Low
Negative weights	Medium	High	High	Reject invalid weights	Low
Incorrect path	Low	High	Medium	Test with known results	Low
Large graph	Medium	Medium	Medium	Limit graph size	Low
Software failure	Low	Medium	Low	Error handling	Very Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project was developed using an iterative software development approach. The navigation network was first represented as a graph, followed by the implementation of BFS, DFS, Dijkstra, and A* algorithms. The system was then tested using different source-destination combinations, and the algorithms were compared based on path distance, explored nodes, and execution time. Finally, the results were visualized and evaluated to identify the most suitable algorithm for different navigation conditions.

Resource	Purpose
Python	Algorithm implementation
Graph data	Navigation network
Computer	Development and testing
Matplotlib	Graph/result visualization

4.2 Software Implementation & Modern Tools Used

Software Implementation

The simulator was implemented in Python using graph data structures. Separate functions were developed for BFS, DFS, Dijkstra, and A* so that each algorithm could be tested independently using the same navigation graph.

Tool / Software	Purpose	Stage Used
Python	Algorithm development	Implementation
Matplotlib	Graph visualization	Results
VS Code / Python IDE	Coding and debugging	Development
CSV	Store test data	Testing

4.3 Test Plan & Verification

Requirement	Test Method	Specification	Required Value	Acceptance Criterion	Achieved Value	Status
Graph creation	Functional test	Valid graph	≥ 10 nodes	Graph created successfully	10+	Pass
BFS	Route test	Valid source/destination	Correct path	Path generated	Achieved	Pass
DFS	Route test	Valid source/destination	Path generated	Path generated	Achieved	Pass
Dijkstra	Weighted test	Non-negative weights	Minimum-cost path	Correct minimum path	Achieved	Pass
A*	Heuristic test	Valid heuristic	Efficient path	Correct path	Achieved	Pass
Distance calculation	Calculation test	Edge weights	Accurate sum	Correct distance	Achieved	Pass
Performance	Timing test	Test graph	≤ 2 seconds	Within limit	Achieved	Pass
Visualization	Display test	Route output	Visible route	Correct display	Achieved	Pass

4.4 Results

The simulator was tested using the same navigation graph and source-destination combinations for all four algorithms.

Sample Simulation Result

Algorithm	Path Found	Distance	Nodes Explored	Time
BFS	Yes	8.5	8	0.001 s
DFS	Yes	11.0	6	0.001 s
Dijkstra	Yes	8.5	9	0.002 s
A*	Yes	8.5	6	0.001 s

4.5 Data Analysis & Engineering Judgment

The results show that Dijkstra and A* produced the minimum-cost route for the tested weighted graph. BFS successfully found a path but is more suitable for unweighted networks, while DFS produced a valid route without guaranteeing minimum distance.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design graph-based navigation network	Graph implemented	Fully
Implement multiple algorithms	BFS, DFS, Dijkstra and A* implemented	Fully
Generate navigation routes	Routes produced for test cases	Fully
Visualize path exploration	Graph visualization implemented	Fully
Measure performance	Distance, nodes and time recorded	Fully
Compare algorithms	Comparative results generated	Fully
Evaluate algorithm suitability	Results analysed	Fully

4.7 Strengths & Limitations

Strengths

- Supports four path-finding algorithms.
- Provides a common environment for comparison.
- Displays explored nodes and final routes.
- Measures distance and execution time.
- Easy to modify and test.

Limitations

- Uses a static graph.
- Does not include real-time traffic.
- Does not use GPS data.
- Performance testing is limited by graph size.
- A* performance depends on heuristic quality.

4.8 Project Planning, Roles & Budget

Project Milestones

Phase	Activity	Duration
1	Literature review and requirements	Week 1
2	Graph design	Week 2
3	Algorithm implementation	Weeks 3–4
4	Visualization	Week 5
5	Testing and debugging	Week 6
6	Results and documentation	Week 7

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The Multi-Algorithm Path Exploration and Navigation Simulator was developed to address the engineering problem of finding and evaluating paths through a connected navigation network. The project represents locations as nodes and connections as edges, with edge weights used to represent distance or travel cost. This graph-based representation provides a suitable environment for applying and comparing different path-finding techniques. The simulator successfully implemented Breadth-First Search (BFS), Depth-First Search (DFS), Dijkstra's algorithm, and A*. Each algorithm was tested using common source and destination locations so that their behaviour could be evaluated under comparable conditions. The system records important performance parameters including the generated route, route distance, number of explored nodes, and execution time. The testing process verified that BFS and DFS can perform general graph exploration, while Dijkstra's algorithm can determine minimum-cost routes in weighted graphs with non-negative edge weights. A* was also able to determine an efficient route by using heuristic information to guide the search. The comparison demonstrated that algorithm performance depends on the structure and characteristics of the navigation network.

5.2 Future Work

Although the current simulator successfully demonstrates multi-algorithm path exploration, several improvements can be introduced in future versions.

1. Real-Time Navigation Data

The simulator can be connected to real-world map and GPS data. This would allow actual geographical locations and road networks to replace the manually created graph.

2. Dynamic Traffic Conditions

Future development could introduce changing traffic conditions by dynamically modifying edge weights. The system could then recalculate routes when congestion or road conditions change.

3. Larger Datasets

The current system can be extended to support large-scale road networks containing thousands or millions of nodes. This would provide a better evaluation of algorithm scalability and computational performance.

4. Advanced A* Heuristics

Different heuristic functions can be investigated to determine how heuristic quality affects A* performance. Geographic distance, estimated travel time, or other cost functions could be evaluated.

5. Interactive User Interface

A web-based or mobile interface could be developed where users can select locations directly on a map, add roads, modify distances, and observe algorithm execution interactively.

6. Multi-Criteria Routing

Future versions could consider multiple factors simultaneously, such as:

- Distance
- Travel time
- Cost
- Energy consumption
- Traffic level

This would make the simulator more representative of practical navigation systems.

7. Dynamic Obstacle Handling

The system could be extended to handle blocked roads, unavailable paths, and temporary obstacles and automatically generate alternative routes.

5.3 Professional Learning & Individual Reflection

5.3.1 New Knowledge Acquired

During the development of the project, the following technical knowledge was acquired:

- Understanding of graph-based representation of real-world navigation networks.

- Detailed understanding of BFS and DFS graph traversal.
- Understanding of weighted shortest-path computation using Dijkstra's algorithm.
- Understanding of heuristic search using A*.
- Knowledge of priority queues and their application in path-finding.
- Understanding of route reconstruction using parent/predecessor nodes.
- Knowledge of measuring execution time and search effort.
- Experience in visualizing graph structures and algorithm results.
- Understanding of algorithmic complexity and its effect on performance.
- Experience in designing repeatable test cases for algorithm comparison.

5.3.2 Engineering & Professional Skills Developed

The project contributed to the development of several engineering and professional skills:

- **Problem-solving:** Breaking a complex navigation problem into graph representation, algorithm, testing, and visualization components.
- **Algorithm design:** Selecting appropriate algorithms based on graph characteristics.
- **Programming:** Implementing and debugging graph algorithms using Python.
- **Testing:** Designing test cases and verifying expected route results.
- **Data analysis:** Comparing algorithms using distance, explored nodes, and execution time.
- **Engineering judgment:** Selecting an appropriate solution based on measurable performance and project constraints.
- **Technical documentation:** Recording requirements, design decisions, testing procedures, and results.
- **Teamwork:** Dividing development, testing, visualization, and documentation activities among team members.
- **Communication:** Presenting technical results in a clear and understandable manner.

REFERENCES

- [1] Y. Liu, X. Gao, B. Wang, J. Fan, Q. Li, and W. Dai, “A passage time–cost optimal A* algorithm for cross-country path planning,” *International Journal of Applied Earth Observation and Geoinformation*, vol. 130, Art. no. 103907, 2024, doi: 10.1016/j.jag.2024.103907.
- [2] Y. Sun, Q. Yuan, Q. Gao, et al., “A multiple environment available path planning based on an improved A* algorithm,” *International Journal of Computational Intelligence Systems*, vol. 17, Art. no. 172, 2024, doi: 10.1007/s44196-024-00571-z.
- [3] J. Ahmad and M. N. Ab Wahab, “Enhancing the safety and smoothness of path planning through an integration of Dijkstra’s algorithm and piecewise cubic Bezier optimization,” *Expert Systems with Applications*, Art. no. 128315, 2025, doi: 10.1016/j.eswa.2025.128315.
- [4] L. Xu, M. Xi, R. Gao, Z. Ye, et al., “Dynamic path planning of UAV with least inflection point based on adaptive neighborhood A* algorithm and multi-strategy fusion,” *Scientific Reports*, vol. 15, Art. no. 8563, 2025.
- [5] “A comprehensive review of path planning algorithms for autonomous navigation,” *Results in Engineering*, Art. no. 107750, 2025, doi: 10.1016/j.rineng.2025.107750.
- [6] “A comprehensive survey of path planning algorithms for autonomous systems and mobile robots: Traditional and modern approaches,” *IEEE Access*, 2025, doi: 10.1109/ACCESS.2025.3618837.
- [7] “Simulation-based review of classical, heuristic, and metaheuristic path planning algorithms,” *Scientific Reports*, 2025.
- [8] “A comprehensive review of improved A* path planning algorithms and their hybrid integrations,” *Applied Sciences*, vol. 6, no. 4, Art. no. 52, 2025.
- [9] X. Zhang, L. Tan, and J. Chai, “Direction aware and self-adaptive A* algorithm with PPO heuristic for UAV path planning of smart city,” *Scientific Reports*, vol. 16, Art. no. 6174, 2026.
- [10] “Intelligent mapping path planning of drones using Dijkstra algorithm,” *Procedia Computer Science*, vol. 281, pp. 1483–1489, 2026, doi: 10.1016/j.procs.2026.05.038.
- [11] “Path planning for autonomous mobile robots in dynamic constrained environments,” *Array*, vol. 30, Art. no. 100906, 2026, doi: 10.1016/j.array.2026.100906.

APPENDICES

APPENDIX C

```
import heapq

import time

import math

# Navigation graph

G = {

    'A': {'B':4, 'C':2},

    'B': {'A':4, 'D':5, 'E':10},

    'C': {'A':2, 'D':8, 'F':4},

    'D': {'B':5, 'C':8, 'E':2, 'F':3},

    'E': {'B':10, 'D':2, 'G':3},

    'F': {'C':4, 'D':3, 'G':6},

    'G': {'E':3, 'F':6}

}

# Coordinates for A* heuristic

P = {

    'A':(0,2), 'B':(2,4), 'C':(2,1),

    'D':(4,2), 'E':(6,4), 'F':(6,1),

    'G':(8,2)

}

# Calculate path distance

def distance(path):

    return sum(G[path[i]][path[i+1]]
```

```

        for i in range(len(path)-1))

# BFS

def bfs(start, goal):

    q = [(start,[start])]

    visited = set()

    explored = []

    while q:

        node, path = q.pop(0)

        if node in visited:

            continue

        visited.add(node)

        explored.append(node)

        if node == goal:

            return path, explored

        for n in G[node]:

            if n not in visited:

                q.append((n,path+[n]))

    return None, explored

# DFS

def dfs(start, goal):

    stack = [(start,[start])]

    visited = set()

    explored = []

```

```

while stack:

    node, path = stack.pop()

    if node in visited:

        continue

    visited.add(node)

    explored.append(node)

    if node == goal:

        return path, explored

    for n in reversed(list(G[node])):

        if n not in visited:

            stack.append((n,path+[n]))

    return None, explored

# Dijkstra

def dijkstra(start, goal):

    pq = [(0,start,[start])]

    visited = set()

    explored = []

    while pq:

        cost,node,path = heapq.heappop(pq)

        if node in visited:

            continue

```

APPENDIX E

Path Exploration
Navigation Simulator

Dashboard

Graph Simulator

Algorithm Comparison

Algorithm Explorer

Saved Graphs

Reports

About

CSA0609 - DAA Capstone

CSA0609 - Design and Analysis of Algorithms

Explore the Path. Understand the Algorithm.

Build a graph, run multiple shortest-path algorithms, visualize their decisions, and compare their performance.

Open Simulator

Explore Algorithms

8 Nodes

12 Edges

A Source

H Destination

4 Algorithms

Quick Start

- Create or load a graph
- Select source and destination
- Choose algorithms
- Run simulation
- Compare results

Start Simulation

Algorithm Overview

Dijkstra
Greedy · Non-negative only
 $O(V + E \log V)$

A*
Heuristic Search · Non-negative only
Depends on heuristic

Bellman-Ford
Repeated Relaxation · Handles negative weights
 $O(V \cdot E)$

Floyd-Warshall
Dynamic Programming · Handles negative weights
 $O(V^3)$

Demo Mode

Path Exploration
Navigation Simulator

Dashboard

Graph Simulator

Algorithm Comparison

Algorithm Explorer

Saved Graphs

Reports

About

CSA0609 - DAA Capstone

GRAPH CONTROLS

+ Add Node

+ Add Edge

Delete

Select

Clear Graph

Reset to Sample

Graph Type

Undirected

Directed

Load Example

Sample City Network

8 nodes, multiple routes — ideal for demos

Small Road Network

6 nodes, simple and clear

Campus Navigation

Campus buildings as nodes

City Network

8 nodes road network

Dense Graph

7 fully-connected nodes

Large Random Graph

10 nodes with random weights

Save Graph

Mode: Select

Dijkstra

A*

Bellman-Ford

Floyd-Warshall

Step 1 / 34

Initialize: distance to source A = 0, all others = ∞.

ALGORITHMS

☒ Dijkstra

☒ A*

☒ Bellman-Ford

☒ Floyd-Warshall

Run Selected

Run All

RESULTS

Dijkstra
0.600 ms · 8 nodes · 12 relax
A → C → D → E → G → H
14

A*
0.400 ms · 5 nodes · 10 relax
A → C → E → G → H
17

Bellman-Ford
0.300 ms · 8 nodes · 48 relax
A → C → D → E → G → H
14

Floyd-Warshall
0.000 ms · 8 nodes · 212 relax
A → C → D → E → G → H
14

ADJACENCY

Matrix

A → B(4) C(2)

B → A(4) C(5) D(10)

C → A(2) B(5) D(3) E(8)

D → A(10) C(1) E(12) F(7)

E → C(8) D(2) F(6) G(4)

F → D(2) E(6) G(5)

G → E(4) F(5) H(3)

Demo Mode

33